LR Parsers

most efficient bottom up Parsing method.

LR(K)

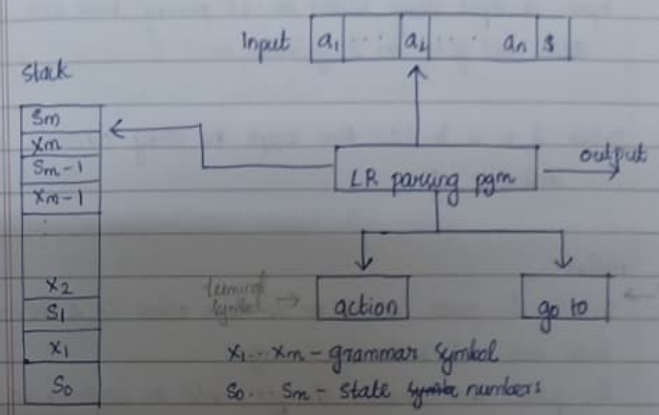
- L → Left to right scanning of the input
- R → Right most derivation in reverse
- K → no. of ip symbols

→ It has 2 parts
- driver routine
- Parsing table

→ 3 types
- driver routine is same
- parsing table construction is different

1. SLR (Simple LR)
   - simple to implement
   - amount of grammer rule it can handle is less
2. CLR (canonical LR)
   - difficult to implement
   - amount of grammar rule it can handle is large
3. LALR Parser (Lookahead LR)
   - amount of grammar rule it can handle is more compared to SLR and less compared to CLR
   - difficulties in implementation is more compared to SLR and less compared to CLR

Model of LR Parsers



Initial configuration

$(S_0 X_1 S_1 X_2 \cdots X_m S_m, a_i a_{i+1} \cdots a_n \$)$

① if action $[S_m, a_i]$ = shift S, then

$(S_0 X_1 S_1 X_2 \cdots X_m S_m a_i S, a_{i+1} \cdots a_n \$)$

where S = action $[S_m, a_i]$

② if action $[S_m, a_i]$ = reduce $A \rightarrow \beta$, then

$(S_0 X_1 S_1 X_2 \cdots X_{m-r} S_{m-r} A S, a_i a_{i+1} \cdots a_n \$)$

where S = goto $[S_{m-r}, A]$ and r is the length of β

③ if action $[S_m, a_i]$ = accept then valid string

④ if action $[S_m, a_i]$ = error then invalid string

15/02/26

LR Parsing Algorithm

Input: An input string w and an LR parsing table with functions action and goto for a grammar G.

Output: If w is in L(G) then accept the string otherwise an error indication.

Method:

Initially the parser has s_0 on its stack where s_0 is the initial state and w$ in the input buffer.

The parser then executes the pgm until an accept or an error action is encountered.

Set input pointer to point to the first symbol of w$

Repeat

Begin

Let s be the state on top of the stack and a the symbol pointed by input pointer

If ~~action of~~ action[s,a] = shift s' then

Begin

push a then s' on top of the stack

advance input pointer to the next input symbol

End

else

if action [s,a] = reduce $A \to \beta$ then

begin

pop 2 * |β| symbols off the stack (→ length of β)

Let s' be the state now on top of the stack

push A then go to [s', A] on top of the stack

output the production $A \to \beta$

End

Else

if action[s,a] = accept then return

Else error()

End

LR(0) item

- Having a dot on the right side of the production

$A \to XYZ$

$A \to .XYZ$

$A \to X.YZ$ } we have verified part of the string by using X and hope that remaining string can be verified using YZ.

$A \to XY.Z$

$A \to XYZ.$

Augmented Grammar

If G is a grammar with start symbol S, then G' - the augmented grammar for G is G' with a new start symbol S' and the production $S' \to S$

Computation of closure

Closure (I), where I is a set of items is computed as follows

1. Add every item in I to closure(I)
2. Repeat for the every item of the form ~~A → α.B~~ $A \to \alpha . B\beta$ in closure(I) do,
   for every production $B \to \gamma$ do,
   add $B \to .\gamma$ to closure(I)
   until no new item can be added to closure(I)

I_0
$A \to \alpha . B\beta$
$B \to .C$
$C \to .b$

Set of items construction

Algorithm to construct C, the canonical collection of sets of item for an Augmented Grammar G', the algo is

Procedure items(G')
begin
C = (closure{S' → .S})
repeat
for each set of items I in C and
for each grammar symbol X such that
goto(I,X) is not empty and not in C do
add goto(I,X) to C, until no more sets of items
can be added to C.

eg: { E → E+T/T
T → T*F/F
F → id, }

closure {E → .E+T}
= {E → .E+T
E → .T
E → .T*F
T → .F
F → .id }

goto(I_0, T) =
closure { E → T.
T → T.*F }
= { E → T.
T → T.*F } I_1 (After taking closure)

Similarly we can generate from I_0 to I_8 for this eg.

20/02/26

E → E+T/T
T → T*F/F
F → id

The Augmented grammar is,
S → E
E → E+T/T
T → T*F/F
F → id

collection of old items ← Cold = ϕ
Consider closure {(S → .E)}
= {S → .E
E → .E+T

$E \to T$

$T \to .T*F$ — I_0

$T \to F$

$F \to .id \}$

$C_{new} = \{I_0\}$

$Temp = \{I_0\}$

$goto(I_0, E) = closure \begin{Bmatrix} S \to E. \\ E \to E+T \end{Bmatrix}$

$= \begin{Bmatrix} S \to E. \\ E \to E.+T \end{Bmatrix}$ — I_1

$goto(I_0, T) = closure \begin{Bmatrix} E \to T. \\ T \to T.*F \end{Bmatrix}$

$= \begin{Bmatrix} E \to T. \\ T \to T.*F \end{Bmatrix}$ — I_2

$goto(I_0, F) = closure \{T \to F.\}$

$= \{T \to F.\}$ — I_3

$goto(I_0, id) = closure \{F \to id.\}$

$= \{F \to id.\}$ — I_4

$goto(I_0, +) = \phi$ → + not preceded with . → so ϕ

$goto(I_0, *) = \phi$

$C_{old} = \{I_0\}$

$C_{new} = \{I_0, I_1, I_2, I_3, I_4\}$

$Temp = \{I_1, I_2, I_3, I_4\}$

I_1 transition

$goto(I_1, E) = \phi$

$goto(I_1, T) = \phi$

$goto(I_1, F) = \phi$

$goto(I_1, id) = \phi$

$goto(I_1, +) = closure \{E \to E+.T\}$

$= \{E \to E+.T$

$T \to .T*F$ — I_5

$T \to .F$

$F \to .id \}$

$goto(I_1, *) = \phi$

I_2 transition

$goto(I_2, E) = \phi$

$goto(I_2, T) = \phi$

$goto(I_2, F) = \phi$

$goto(I_2, id) = \phi$

$goto(I_2, +) = \phi$

$goto(I_2, *) = closure \{T \to T*.F\}$

$= \{T \to T*.F$ — I_6

$F \to .id \}$

I_3 transition

$goto(I_3, E) = \phi$

$goto(I_3, T) = \phi$

$goto(I_3, F) = \phi$

$goto(I_3, id) = \phi$

$goto(I_3, +) = \phi$

$goto(I_3, *) = \phi$

I_4 transition

$goto(I_4, E) = \phi$

$goto(I_4, T) = \phi$

$goto(I_4, F) = \phi$

$goto(I_4, id) = \phi$

$goto(I_4, +) = \phi$

$goto(I_4, *) = \phi$

$C_{old} = \{I_0, I_1, I_2, I_3, I_4\}$

$C_{new} = \{I_0, I_1, I_2, I_3, I_4, I_5, I_6\}$

$Temp = \{I_5, I_6\}$

I_5 transition

$goto(I_5, E) = \phi$

$goto(I_5, T) = closure\left\{\begin{matrix} E \to E + T. \\ T \to T. * F \end{matrix}\right\}$

$= \left\{\begin{matrix} E \to E + T. \\ T \to T. * F \end{matrix}\right\}$ - I_7

$goto(I_5, F) = closure\{T \to F.\}$

$= \{T \to F.\}$ - I_3

$goto(I_5, id) = closure\{F \to id.\}$

$= \{F \to id.\}$ - I_4

$goto(I_5, +) = \phi$

$goto(I_5, *) = \phi$

I_6 transition

$goto(I_6, E) = \phi$

$goto(I_6, T) = \phi$

$goto(I_6, F) = closure\{T \to T * F.\}$

$= \{T \to T * F.\}$ - I_8

$goto(I_6, id) = closure\{F \to id.\}$

$= \{F \to id.\}$ - I_4

$goto(I_6, +) = \phi$

$goto(I_6, *) = \phi$

$C_{old} = \{I_0, I_1, I_2, \cdots \quad I_8\}$

$C_{new} = \{I_0, I_1, I_2, I_3, I_4, I_5, I_6, I_7, I_8\}$

$Temp = \{I_7, I_8\}$

I_7 transition

$goto(I_7, E) = \phi$

$goto(I_7, T) = \phi$

$goto(I_7, F) = \phi$

$goto(I_7, id = \phi$

$goto(I_7, +) = \phi$

$goto(I_7, *) = closure\{T \to T * . F\}$

$= \left\{\begin{matrix} T \to T * . F \\ F \to . id \end{matrix}\right\}$ - I_6

I_8 transition

$goto(I_8, E) = \phi$

$goto(I_8, T) = \phi$

$goto(I_8, F) = \phi$

$goto(I_8, id) = \phi$

$goto(I_8, +) = \phi$

$goto(I_8, *) = \phi$

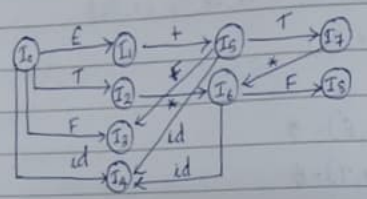
$C_{old} = \{I_0, I_1, \cdots I_8\}$

$C_{new} = \{I_0, I_1, \cdots I_8\}$

$Temp = \phi$

2/03/2026

The transition diagram



~~Construction~~

Construction of action table

i) For every state I_i in C do, for every terminal a, do
if goto $I_i, a = I_j$, then make action $[I_i, a]$ = shift j

ii) For every state I_i in C whose indentined set of items contain an item of the ~~form~~ form $a \to \alpha.$ do for every b in ~~follow of a~~ follow(A) do
make action $[I_i, b] = R_k$, where k is no. of the production $A \to \alpha$
standing for reducing by $A \to \alpha$
here A may not be S'.

iii) make $[I_i, \$]$ = accept if I_i contains an item $S' \to S$

Construction of goto table:
A goto table is a simple mapping of transition in the DFA on non terminals. It is
for every I_i in C do
for every non terminal A do
if goto $(I_i, A) = I_j$ then
make goto $(I_i, A) = j$

Parsing table

States	Action				Goto		
	id	+	*	$	E	T	F
I_0	S4				1	2	3
I_1		S5		accept			
I_2		R2	S6	R2			
I_3		R4	R4	R4			
I_4		R5	R5	R5			
I_5	S4				7	3	
I_6	S4					8	
I_7		R1	S6	R1			
I_8		R3	R3	R3			

$E \to E+T$ – R1 Follow(E) = {$, +}
$E \to T$ – R2 Follow(T) = {*, +, $,)}
$T \to T*F$ – R3 Follow(F) = {*, $, +,)}
$T \to F$ – R4
$F \to id$ – R5

($(I_1, \$)$ → accept by rule 3. I_1 has a production with in the extreme right & it has left side with new str, & right with old start symbol)

03/03/26

Stack	Input	Action
0	id * id + id $	shift
0 id 4	* id + id $	Reduce F → id
0 F 3	* id + id $	Reduce T → F

length, r = 1
|rp| = 2r = 2×1 = 1

0T2	*id+id$	Shift
0T2*6	id+id$	Shift
0T2*6id4	+id$	Reduce F→id
0T2*6F8	+id$	Reduce T→T*F
0T2	+id$	Reduce E→T
0E1	+id$	Shift
0E1+5	id$	Shift
0E1+5id4	$	Reduce F→id
0E1+5F3	$	Reduce T→F
0E1+5T7	$	Reduce E→E+T
0E1	$	accept

eg: Create parsing table

$S \to AaAb$

$S \to BbBa$

$A \to \varepsilon$

$B \to \varepsilon$

Augmented Grammar is,

$S' \to S$

$S \to AaAb$

$S \to BbBa$

$A \to \varepsilon$

$B \to \varepsilon$

$C_{old} = \phi$

consider closure $\{(S' \to .S)\}$

$= \{S' \to .S$

$S \to .AaAb$ — I_0

$S \to .BbBa$

$A \to .$

$B \to . \}$

$A \to .\varepsilon$ is $A \to .$

$A \to \varepsilon$. is $A \to .$

$C_{new} = \{I_0\}$

$Temp = \{I_0\}$

$goto(I_0, S) = closure\{S' \to S.\}$

$= \{S' \to S.\}$ — I_1

$goto(I_0, A) = closure\{S \to A.aAb\}$

$= \{S \to A.aAb\}$ — I_2

$goto(I_0, B) = closure\{S \to B.bBa\}$

$= \{S \to B.bBa\}$ — I_3

$goto(I_0, a) = \phi$

$goto(I_0, b) = \phi$

$C_{old} = \{I_0\}$, $C_{new} = \{I_0, I_1, I_2, I_3\}$, $Temp = \{I_1, I_2, I_3\}$

I_1 transition

$goto(I_1, S) = \phi$

$goto(I_1, A) = \phi$

$goto(I_1, B) = \phi$

$goto(I_1, a) = \phi$

$goto(I_1, b) = \phi$

I_2 transition

$goto(I_2, S) = \phi$

$goto(I_2, A) = \phi$

$goto(I_2, B) = \phi$

$goto(I_2, a) = closure\{S \to Aa.Ab\}$

$= \{S \to Aa.Ab$

$A \to .\}$ — I_4

$goto(I_2, b) = \phi$

I_3 transition

goto$(I_3, S) = \phi$

goto$(I_3, A) = \phi$

goto$(I_3, B) = \phi$

goto$(I_3, a) = \phi$

goto(I_3, b) = closure$\{S \to Bb.Ba\}$

$= \{S \to Bb.Ba$ - I_5

$B \to .\}$

I_4 transition

goto$(I_4, S) = \phi$

goto(I_4, A) = closure$\{S \to AaA.b\}$

~~goto~~ $= \{S \to AaA.b\}$ - I_6

goto$(I_4, B) = \phi$

goto$(I_4, a) = \phi$

goto$(I_4, b) = \phi$

upto I_3 → Cold = $\{I_0, I_1, I_2, I_3\}$

Cnew = $\{I_0, I_1, I_2, I_3, I_4, I_5\}$

Temp = $\{I_4, I_5\}$

I_5 transition

goto$(I_5, S) = \phi$

goto$(I_5, A) = \phi$

goto(I_5, B) = closure$\{S \to BbB.a\}$

$= \{S \to BbB.a\}$ - I_7

goto$(I_5, a) = \phi$

goto$(I_5, b) = \phi$

I_6 transition

goto$(I_6, S) = \phi$

goto$(I_6, A) = \phi$

goto$(I_6, B) = \phi$

goto$(I_6, a) = \phi$

goto(I_6, b) = closure$\{S \to AaAb.\}$

$= \{S \to AaAb.\}$ - I_8

I_7 transition

goto$(I_7, S) = \phi$

goto$(I_7, A) = \phi$

goto$(I_7, B) = \phi$

goto(I_7, a) = closure$\{S \to BbBa.\}$

$= \{S \to BbBa.\}$ - I_9

goto$(I_7, b) = \phi$

Cold = $\{I_0, I_1, I_2, I_3, I_4, I_5, I_6, I_7\}$

Cnew = $\{I_0, I_1, I_2, I_3, I_4, I_5, I_6, I_7, I_8, I_9\}$

Temp = $\{I_8, I_9\}$

I_8 transition

goto$(I_8, S) = \phi$

goto$(I_8, A) = \phi$

goto$(I_8, B) = \phi$

goto$(I_8, a) = \phi$

goto$(I_8, b) = \phi$

I_9 transition

goto$(I_9, S) = \phi$

goto$(I_9, A) = \phi$

goto$(I_9, B) = \phi$

goto$(I_9, a) = \phi$

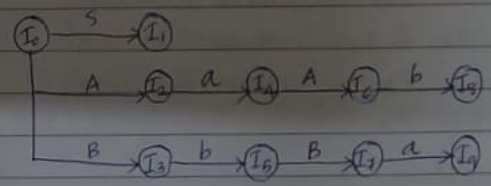
goto$(I_9, b) = \phi$

Cold = $\{I_0, I_1, I_2, I_3, I_4, I_5, I_6, I_7, I_8, I_9\}$

Cnew = $\{I_0, I_1, I_2, I_3, I_4, I_5, I_6, I_7, I_8, I_9\}$

Temp = ϕ, hence we can stop the iteration

Transition dgm



Parsing table

State	Action			goto		
	a	b	$	S	A	B
I_0	R3 R4	R4 R3		1	2	3
I_1			accept			
I_2	S4					
I_3		S5				
I_4	R3	R3			6	
I_5	R4	R4				7
I_6		S8				
I_7	S9					
I_8			R1			
I_9			R2			

number the rules

$S \to AaAb$ R_1 FOLLOW(S) = {$}

$S \to BbBa$ R_2 FOLLOW(A) = {a, b}

$A \to \varepsilon$ R_3 FOLLOW(B) = {a, b}

$B \to \varepsilon$ R_4

04/03/26 CLR

LR(1) item has 2 parts

LR(0) item — Lookahead

function closure(I)

begin

repeat

for each item $[A \to \alpha \cdot B\beta, a]$ in I

each production $B \to \gamma$ in G'

and each terminal b in FIRST(βa)

such that $[B \to \cdot\gamma, b]$ is not in I do

add $[B \to \cdot\gamma, b]$ to I until no more items can be added to I.

return I.

end

Procedure items(G')

begin

C = {closure{$S' \to \cdot S, \$$}}

repeat

for each set of items I in C

and each grammar symbol x such that

goto[I, x] is not empty and not in C do

add goto[I, x] to C until no more sets of items can be added to C

end

09/03/26 eg: $S \to CC$

$C \to cC/d$

The augmented grammar is

$S' \to S$

$S \to CC$

$C \to cC/d$

closure $\{S' \to .S, \$\}$

$\{S' \to .S(\$)$

$S \to .CC, \$$ I_0

$C \to .cC, c/d$

$C \to .d, c/d\}$

goto(I_0, S) = closure $\{S' \to S., \$\}$

$= \{S' \to S., \$\}$ I_1

goto(I_0, C) = closure $\{S \to C.C, \$\}$

$= \{S \to C.C, \$$

$C \to .cC, \$$ I_2

$C \to .d, \$\}$

goto(I_0, c) = closure $\{C \to c.C, c/d\}$

$= \{C \to c.C, c/d$

$C \to .cC, c/d$ I_3

$C \to .d, c/d\}$

goto(I_0, d) = closure $\{C \to d., c/d\}$

$= \{C \to d., c/d\}$ I_4

temp $= I_1, I_2, I_3, I_4$

~~goto~~ No transition for I_1

goto$(I_2, S) = \phi$

goto(I_2, C) = closure $\{S \to CC., \$\}$

$= \{S \to CC., \$\}$ I_5

goto(I_2, c) = closure $\{C \to c.C, \$\}$

$= \{C \to c.C, \$$

$C \to .cC, \$$ I_6

$C \to .d, \$\}$

goto(I_2, d) = closure $\{C \to d., \$/\cancel{c/d}\}$

$= \{C \to d., \$/\cancel{c/d}\}$ I_7

goto(I_3, C) = closure $\{C \to cC., c/d\}$

$= \{C \to cC., c/d\}$ I_8

goto(I_3, c) = Closure $\{C \to c.C, c/d\}$

$= \{C \to c.C, c/d$

$C \to .cC, c/d$ I_3

$C \to .d, c/d\}$

goto(I_3, d) = closure $\{C \to d., c/d\}$

$= \{C \to d., c/d\}$ – I_4

No transition for I_4 and I_5

Temp $= \{I_5, I_6, I_7, I_8\}$

goto(I_6, C) = closure $\{C \to cC., \$\}$

$= \{C \to cC., \$\}$ I_9

goto(I_6, c) = Closure $\{C \to c.C, \$\}$

$= \{C \to c.C, \$$

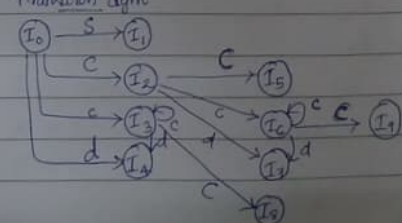
$C \to .cC, \$$ I_6

$C \to .d, \$\}$

goto(I_6, d) = closure $\{C \to d., \$\}$

$= \{C \to d., \$\}$ I_7

Transition dgm



Construction of CLR parsing table

I/p: Augmented Grammar G'

O/p: CLR Parsing table

Method:

1. Construct $C = \{I_0, I_1, \cdots I_n\}$, the collection of sets of canonical LR1 items for G'.
2. a. if $[A \to \alpha . a\beta, b]$ is in I_i, and $goto[I_i, a] = I_j$ then
   set action$[i, a]$ to shift j
   here a is a terminal
   b. if $[A \to \alpha ., a]$ is in I_i,
   $A \neq S'$ then set action$[i, a]$ to reduce $A \to \alpha$
   c. if $[S' \to . S, \$]$ is in I_i,
   then set action$[\$i, \$]$ to
   accept
3. If $goto[I_i, A] = I_j$ then
   $goto[i, A] = j$
4. All entries not defined by above rules are
   made error

$S \to CC$ R_1

$C \to cC$ R_2

$C \to d$ R_3

Parse table

States	Action			Goto	
	c	d	$	S	C
I_0	S_3	S_4		1	2
I_1			Accept		
I_2	S_6	S_7			5
I_3	S_3	S_4			8
I_4	R_3	R_3			
I_5			R_1		
I_6	S_6	S_7			9
I_7			R_3		
I_8	R_2	R_2			
I_9			R_2		

HW

eg: $S \to AaAb$

$S \to BbBa$

$A \to \varepsilon$

$B \to \varepsilon$

Ans: Augmented Grammar

$S' \to S$

$S \to AaAb$

$S \to BbBa$

$A \to \varepsilon$

$B \to \varepsilon$

item construction

closure $\{S' \to .S, \$\} = \{S' \to .S, \$$

$S \to .AaAb, \$$

$S \to .BbBa, \$$ $\quad I_0$

$A \to ., a$

$B \to ., b\}$

$C_{old} = \phi$, $C_{new} = I_0$, $Temp = \{I_0\}$

goto(I_0, S) = closure $\{S' \to S., \$\}$

$= \{S' \to S., \$\}$ - I_1

goto(I_0, A) = closure $\{S \to A.aAb, \$\}$

$= \{S \to A.aAb, \$\}$ - I_2

goto(I_0, B) = closure $\{S \to B.bBa, \$\}$

$= \{S \to B.bBa, \$\}$ $\quad I_3$

goto$(I_0, a) = \phi$

goto$(I_0, b) = \phi$

$C_{old} = I_0$ $\quad C_{new} = \{I_0, I_1, I_2, I_3\}$ $\quad Temp = \{I_1, I_2, I_3\}$

no transition for I_1

goto(I_2, a) = closure $\{S \to Aa.Ab, \$\}$

$= \{S \to Aa.Ab, \$$ $\quad I_4$

$A \to ., b\}$

goto$(I_2, b) = \phi$

goto(I_3, b) = closure $\{S \to Bb.Ba, \$\}$

$= \{S \to Bb.Ba, \$$ $\quad I_5$

$B \to ., a\}$

$C_{old} = \{I_0, I_1, I_2, I_3\}$ $\quad C_{new} = \{I_0, I_1, I_2, I_3, I_4, I_5\}$

$Temp = \{I_4, I_5\}$

goto(I_4, A) = closure $\{S \to AaA.b, \$\}$

$= \{S \to AaA.b, \$\}$ - I_6

goto(I_5, B) = closure $\{S \to BbB.a, \$\}$

$= \{S \to BbB.a, \$\}$ - I_7

$C_{old} = \{I_0, I_1, I_2, I_3, I_4, I_5\}$ $\quad C_{new} = \{I_0, I_1, I_2, I_3, I_4, I_5, I_6, I_7\}$

$Temp = \{I_6, I_7\}$

goto(I_6, b) = closure $\{S \to AaAb., \$\} = \{S \to AaAb., \$\}$ - I_8

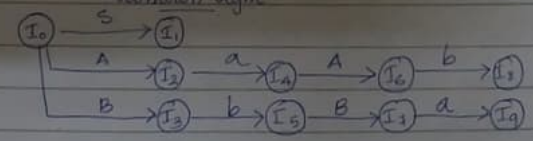
goto(I_7, a) = closure $\{S \to BbBa., \$\} = \{S \to BbBa., \$\}$ - I_9

no transition for I_8 and I_9

$C_{old} = \{I_0, I_1, \ldots, I_8, I_9\}$ $\quad C_{new} = \{I_0, I_1, \ldots, I_7, I_8, I_9\}$

$Temp = \phi$

transition dgm



$S \to AaAb$ $\quad R_1$

$S \to BbBa$ $\quad R_2$

$A \to \varepsilon$ $\quad R_3$

$B \to \varepsilon$ $\quad R_4$

Parse table

State	Action			Goto		
	a	b	$	S	A	B
I_0	R_3	R_4		1	2	3
I_1			Accept			
I_2	S_4					
I_3		S_5				
I_4		R_3			6	
I_5	R_4					7
I_6		S_8				
I_7	S_9					
I_8			R_1			
I_9			R_2			

18/03/26

LALR Parser

combine the items which are having similar LR(0), but different lookahead.

Construction of Action table

1. for every state I_i in C do
   for every terminal symbol 'a' do
   if goto $(I_i, a) = I_j$ then
   make action $[I_i, a] = S_j$
2. for every state I_i in C whose set of LR_1 items contains an item of the form $A \to \alpha \cdot, a$ do
   make action $[I_i, a] = R_k$ where k is the no. of the production $A \to \alpha$ standing for reduce by $A \to \alpha$
3. Make action $[I_i, \$] =$ accept
   if I_i contain an item $S' \to S, \$$

Construction of goto table

1. for every I_i in C do
   for every nonterminal 'A' do
   if goto $(I_i, A) = I_j$ then
   make goto $(I_i, A) = j$

solving prev eg:

$S \to CC$

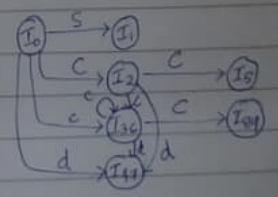
$C \to cC \mid d$

I_0, I_1, I_2, I_5

$I_{36} = \{ C \to c \cdot C, c/d/\$$
$C \to \cdot cC, c/d/\$$
$C \to \cdot d, c/d/\$ \}$

$I_{47} = \{ C \to d \cdot, c/d/\$ \}$

$I_{89} = \{ C \to cC \cdot, c/d/\$ \}$



Parsing table

state	Action			Goto	
	c	d	$	S	C
I_0	S_{36}	S_{47}		1	2
I_1			Accept		
I_2	S_{36}	S_{47}			5
I_{36}	S_{36}	S_{47}			89
I_{47}	R_3	R_3	R_3		
I_5			R_1		
I_{89}	R_2	R_2	R_2		

YACC (Yet Another Compiler Compiler)

* Parser Generator
* Linux environment
* YACC has 3 parts
  - declarations — normal declaration, Regular definitions
    %%
  - translation rules — <left side> <alt 1> {semantic action 1}
    <alt 2> {semantic action 2}
    %%
    <alt n> {semantic action n}
  - Supporting C routines — auxiliary procedure

sample.y → [YACC] → y.tab.c

y.tab.c → [C compiler] → a.out

Parser for $E \rightarrow E+E \mid E*E \mid id$

```
id [a-zA-Z] [a-zA-Z 0-9]*
%%
{id}    {return id ;}
%%
% start P
% token id
% left "+" "-"
% left "*" "/"
%%
P: E ;
E: E '+' E {printf("the expn is of the form E+E");}
 | E '*' E {printf("the expn is of the form E*E");}
 | id {printf("identifier");}
%%
void main()
{ yyparse();
}
```